

Learning-to-Rank with Pairwise Optimization

In this task, we shift from heuristic ranking to a learned ranking model. We selected Bayesian Personalized Ranking (BPR) with matrix factorization because it is the standard pairwise learning-to-rank method for implicit feedback. It directly optimizes the order of items rather than predicting ratings, which is the main goal of a recommender system.

A pointwise approach would attempt to predict whether a user will interact with an item, which is a binary classification problem. The problem is that we do not actually have negative labels. A user not watching a movie does not mean they dislike it; it just means they haven't seen it yet. BPR avoids this issue by requiring only that observed interactions rank above unobserved ones, which is a weaker and more realistic assumption.

Implementation

For each positive pair (u, i) , we choose a negative item j randomly from items user u has not interacted with, ensuring it's a true negative. We opted for uniform sampling which has a downside, which is evident in our results, is that many sampled negatives are obscure items that the model already ranks low. This means many gradient updates are essentially wasted. We discuss this further in the sensitivity section below.

We apply L2 regularization ($\lambda = 0.01$) to all three sets of embeddings updated at each step: the user vector, the positive item vector, and the negative item vector. Without regularization, the embeddings tend to grow without limit, especially for users or items with few interactions, leading the model to overfit quickly. We use a single λ for simplicity. In principle, you could fine-tune separate values for users and items, but in this dataset, one coefficient was enough to prevent overfitting.

The entire training loop, including gradients, sampling, and parameter updates, is written from scratch in NumPy, without any external ML library.

Results

Ranking metrics

Model	NDCG@10	MAP@10	Precision@10	Recall@10
Popularity	0.0639	0.0295	0.0541	0.0432
BPR	0.0774	0.0359	0.0641	0.0556

BPR outperforms popularity in every metric, though the difference is not large. A natural question arises about whether these numbers are "good enough." Published BPR results on MovieLens 1M often report higher NDCG values. The difference likely stems from our use of only one negative sample per positive in the main run, to keep training fast for convergence analysis, and from our relatively conservative hyperparameters. The sampling sweep below indicates that increasing negatives to 10 boosts validation NDCG to 0.118, a more significant improvement.

Beyond-accuracy metrics

Model	Coverage	Novelty
-------	----------	---------

Model	Coverage	Novelty
Popularity	3.1%	8.37
BPR	19.3%	8.77

This is where the difference stands out. Popularity recommends about 110 items out of ~3,700, focusing only on the same blockbusters for everyone. BPR, on the other hand, suggests over six times more unique items by learning user embeddings that allow it to surface different items for different people. This is important in practice. A system with 3% coverage essentially ignores 97% of its catalog.

Convergence behavior

Training loss decreases smoothly over all 50 epochs, without spikes or instability. Validation NDCG@10 rises quickly in the first 10 to 15 epochs and then levels off, indicating that the model reaches a reasonable solution early on. There's no gap between training loss continuing to drop and validation metrics degrading, so overfitting isn't an issue here, L2 regularization is working well.

Why does it converge quickly at first? Early in training, nearly every sampled negative is an "easy" item that the model scores well below the positive. The gradient from these easy negatives is strong and consistent, allowing the embeddings to improve rapidly. As training progresses, the remaining gains come from "harder" negatives, items closer to the decision boundary, which provide smaller, noisier gradients. This explains the diminishing returns per epoch.

Sensitivity to negative sampling

Negative Samples	NDCG@10	Recall@10	Training Time (s)
1	0.0801	0.0574	101
3	0.1060	0.0809	189
5	0.1036	0.0791	316
10	0.1178	0.1066	616

More negatives typically help, but the relationship is not perfectly linear. Five negatives perform slightly worse than three on this validation split. This is not entirely surprising; with a fixed learning rate and epoch count, doubling the negatives does not simply double the signal. It also alters the effective gradient size per step, which can negatively affect the learning rate. A proper hyperparameter search, tuning the learning rate alongside negative count, would likely improve this situation.

The most significant improvement occurs when moving from one to three negatives. Increasing from three to ten adds about a 10% relative improvement but requires over three times longer to train. In a production setting, three negatives would likely be the sweet spot because the extra quality from ten may not justify the additional computational cost.

Why does uniform sampling hit diminishing returns so quickly? In a catalog of ~3,700 items, most are unpopular, and the model learns to rank these below positives almost immediately. Additional random samples tend to come from this already-solved pool. Smarter strategies, like popularity-biased sampling or hard negative mining (selecting items the model ranks closely to the positive), would focus gradient signals

where they are most needed. However, these methods would add complexity to the training loop and may introduce instability.

Head vs. tail items

Model	Segment	NDCG@10	Recall@10
Popularity	all	0.0639	0.0432
Popularity	head	0.0699	0.0573
Popularity	tail	0.0000	0.0000
BPR	all	0.0774	0.0556
BPR	head	0.0857	0.0772
BPR	tail	0.0007	0.0005

We divided items into head (top 20% by training interactions) and tail (remaining 80%). The results are striking: both models essentially fail on tail items. Popularity scores zero, as it only recommends top items by design. BPR achieves an NDCG of 0.0007, which is technically non-zero but practically useless.

Why does BPR fail on the tail despite being a personalized model? Two reinforcing factors explain this. First, head items appear much more frequently in the training data, so their embeddings are updated more often and become better calibrated. Second, uniform negative sampling works against tail items, when a random negative is selected, it is almost always a tail item. This leads the model to push tail items down instead of learning meaningful distinctions among them. BPR's personalization mainly helps within the head: it identifies which popular movies a user would prefer but rarely brings forward niche content.

Overall comparison

Popularity is a surprisingly strong baseline. It has no parameters, trains instantly, and is entirely deterministic. In terms of aggregate metrics, it performs well because the test set is mainly made up of interactions with popular items, which happen to be what popularity recommends.

BPR improves on popularity by learning personalized preferences through pairwise ranking. The benefits appear in two areas: better ranking among top items (it identifies which popular items each user likes, rather than just noting that some items are popular) and significantly higher catalog coverage (19.3% versus 3.1%). However, the downside is significant; training takes much longer, and there are several hyperparameters to adjust, including embedding dimension, learning rate, regularization, batch size, and negative samples.

The key point is that BPR with uniform sampling and careful hyperparameters offers a modest improvement over popularity. The model achieves its goal of personalizing rankings, but it does not address the long-tail problem. To tackle that issue, you would need either a different sampling approach or extra signals beyond collaborative filtering.